

```

1  /*
2  *  RÉGULATEUR DE VITESSE ADAPTATIF ET SA BOUCLE DE RÉTROACTION PID
3  *  TIPE Par Emile Reynaud
4  */
5
6  // Définition de 4 constantes numériques au lieu d'avoir des str
7  #define CORR_P 1 // correcteur proportionnel seul
8  #define CORR_PI 2 // correcteur proportionnel + intégral
9  #define CORR_PD 3 // correcteur proportionnel + dérivé
10 #define CORR_PID 4 // correcteur proportionnel + intégral + dérivé
11
12 // Choix du correcteur en décommentant une seule ligne
13 // #define CORRECTEUR CORR_P
14 // #define CORRECTEUR CORR_PI
15 // #define CORRECTEUR CORR_PD
16 #define CORRECTEUR CORR_PID
17
18 // Définition des bonnes constantes en fonction du correcteur
19 #if CORRECTEUR == CORR_P
20   #define Kp_VAL 60.0f
21   #define Ki_VAL 0.0f
22   #define Kd_VAL 0.0f
23   #define USE_I 0 // activation ou non de l'intégrale
24   #define USE_D 0 // idem pour la dérivée
25   #define MODE_STR "P" // label pour logs
26   #define KICK_PWM_VAL 209 // PWM du kick
27   #define KICK_DUREE_MS_VAL 250 // durée du kick
28
29 #elif CORRECTEUR == CORR_PI
30   #define Kp_VAL 15.0f
31   #define Ki_VAL 60.0f
32   #define Kd_VAL 0.0f
33   #define USE_I 1
34   #define USE_D 0
35   #define MODE_STR "PI"
36   #define KICK_PWM_VAL 209
37   #define KICK_DUREE_MS_VAL 250
38
39 #elif CORRECTEUR == CORR_PD
40   #define Kp_VAL 350.0f
41   #define Ki_VAL 0.0f
42   #define Kd_VAL 10.0f
43   #define USE_I 0
44   #define USE_D 1
45   #define MODE_STR "PD"
46   #define KICK_PWM_VAL 255
47   #define KICK_DUREE_MS_VAL 800
48
49 #elif CORRECTEUR == CORR_PID
50   #define Kp_VAL 80.0f
51   #define Ki_VAL 40.0f
52   #define Kd_VAL 8.0f
53   #define USE_I 1
54   #define USE_D 1
55   #define MODE_STR "PID"
56   #define KICK_PWM_VAL 153
57   #define KICK_DUREE_MS_VAL 1000
58
59 #else
60   // Si pas de correcteur valide, renvoie une erreur
61   #error "Correcteur non défini – décommenter une ligne CORRECTEUR ci-dessus"
62 #endif
63
64 // Identification moteur
65 const float K_MOTEUR = 1.40f; // gain statique du moteur
66 const float SEUIL_DEMARRAGE = 0.30f; // commande minimale normalisée pour démarrer
67
68 // Déclaration de la variable du feedforward
69 // Calcul de cette dernière dans setup()
70 float FEEDFORWARD_PWM = 0.0f;
71
72 // Gain de freinage pour le mode distance
73 const float KP_BRAKE_PWM = 600.0f; // en PWM/m
74
75 // Pins Arduino
76 #define TRIG_PIN 7 // pin TRIG du capteur à ultrason
77 #define ECHO_PIN 6 // pin ECHO du capteur à ultrason
78 #define HALL_PIN 2 // pin SIG du capteur à effet Hall
79 #define ENB_PIN 9 // pin ENB du pont en H
80 #define IN3_PIN A0 // pin IN3 du pont en H
81 #define IN4_PIN 5 // pin IN4 du pont en H
82 #define SD_CS_PIN 4 // pin SD_CS de la carte microSD
83 #define BUTTON_PIN 8 // bouton démarrage programme
84
85 // Roue
86 const int DIAMETRE_MM = 83; // diamètre de la roue (mm)
87 const float PERIMETRE_M = 3.14159f * (float)DIAMETRE_MM / 1000.0f; // Périmètre de la roue (m)
88 const uint8_t NB_AIMANTS = 3; // nombre d'aimants
89
90 // Constantes de consigne
91 const float DIST_CONSIGNE_M = 0.30f; // distance de consigne (m)
92 const float VITESSE_CIBLE_MS = 0.70f; // vitesse de consigne (m/s)
93
94 // Constantes de sécurité
95 const float DIST_URGENCE_M = 0.06f; // distance freinage d'urgence
96 const float DIST_MAX_OBSACLE_M = 1.20f; // distance max à laquelle un objet est détecté
97
98 // Limite de l'intégrale
99 const float INTEGRALE_MAX = 100.0f; // valeur maximale de l'intégral (en PWM)
100
101 // Constantes temporelles
102 const uint16_t CONTROL_PERIOD_MS = 50; // période de la boucle de contrôle
103 const uint16_t LOG_PERIOD_MS = 100; // période d'écriture sur la carte SD

```

```

104 const uint16_t DEBOUNCE_MS = 30; // durée de l'anti-rebond du bouton
105 const uint32_t ECHO_TIMEOUT_US = 12000UL; // timeout capteur à ultrason
106 const uint32_t HALL_TIMEOUT_US = 40000UL; // timeout capteur à effet Hall
107
108 // Constantes pour le kick start
109 const uint8_t KICK_PWM_C = KICK_PWM_VAL; // puissance du kick
110 const uint16_t KICK_DUREE_MS_C = KICK_DUREE_MS_VAL; // durée du kick (ms)
111 const float VITESSE_QUASI_NULLE_MS = 0.05f; // en dessous de cette vitesse, on considère la roue arrêtée
112 const float COMMANDE_MIN_DEMARRAGE = 3.0f; // commande minimale pour déclencher un kick (PWM)
113
114 // Zone morte
115 const float PWM_ZONE_MORTE = 76.0f;
116
117 // Rampe de commande (lissage asymétrique)
118 const float RAMPE_MONTEE_PAR_S = 350.0f; // PWM/s accélération
119 const float RAMPE_DESCENTE_PAR_S = 700.0f; // PWM/s décélération
120
121 // Bibliothèques
122 #include <SPI.h> // protocole SPI nécessaire pour la carte SD
123 #include <SD.h> // permet de lire/écrire sur la carte SD
124
125 // Variables volatiles (pour éviter que le programme les mettent en cache car elle sont constamment modifiées)
126 volatile uint32_t hallPulseCount = 0; // nombre d'impulsions Hall reçues
127 volatile uint32_t hallLastPulseTime_us = 0; // timestamp (microseconde) de la dernière impulsion Hall
128 volatile uint32_t hallPeriode_us = 0; // durée (microseconde) entre les deux dernières impulsions
129 volatile bool hallNouvelleImpulsion = false; // nouvelle impulsion ou non
130
131 // Variables de mesures
132 uint32_t lastHallCount = 0; // valeur de hallPulseCount au dernier cycle (méthode fréquentométrique)
133 float vitesse_ms = 0.0f; // vitesse mesurée par méthode fréquentométrique (m/s)
134 float vitesse_periode_ms = 0.0f; // vitesse mesurée par méthode période (m/s)
135
136 // Variables correcteur
137 float integrale = 0.0f; // accumulateur de l'intégrale (terme I du PI et PID)
138
139 // Dérivée calculée sur la mesure de vitesse
140 float vitessePeriodePrecedente = 0.0f; // vitesse_periode_ms du cycle précédent
141 bool deriveeInitialisee = false; // faux au démarrage, le premier cycle n'a pas de "précédent"
142
143 // Variables temporelles
144 uint32_t lastControlTime = 0; // timestamp du dernier contrôle (ms)
145 uint32_t lastLogTime = 0; // timestamp du dernier log SD (ms)
146 float derniereCommande = 0.0f; // PWM envoyée au moteur au dernier cycle
147
148 // Variables kick start
149 bool kickActif = false; // kick actif ou non
150 uint32_t kickDebutMs = 0; // timestamp (ms) du début du kick en cours
151 float commandeLisseePWM = 0.0f; // commande lissée par la rampe
152
153 // Variables carte SD
154 File logFile; // objet fichier ouvert sur la carte SD
155 bool sdOk = false; // carte SD initialisée et fichier ouvert ou non
156 char logFilename[13] = {0}; // nom du fichier CSV
157 uint16_t logLinesSinceFlush = 0; // compteur de lignes depuis le dernier flush() forcé
158 const uint8_t LOG_FLUSH_EVERY_N_LINES = 10; // on force un flush tous les 10 lignes pour ne pas perdre trop de données en cas de coupure
159
160 // Variables bouton
161 bool programmeActif = false; // régulateur actif ou non
162 const uint8_t BUTTON_ACTIVE_LEVEL = LOW; // état du pin quand le bouton est pressé
163 int buttonLastRawState = HIGH; // état lu au cycle précédent
164 int buttonStableState = HIGH; // état stable après anti-rebond
165 uint32_t lastButtonChangeTime = 0; // timestamp (ms) du dernier changement d'état brut
166
167 // Déclarations anticipées des fonctions
168 void appliquerCommande(float cmd);
169 void ecrireLog(float t_s, float dist_m, float erreur_m, uint8_t modeActuel);
170 void initSD();
171 void fermerLogSD();
172 float calculerCommandeMoteur(float cmdCorrecteur, float dt_s, bool urgence, bool interdireKick, uint32_t now);
173 float calculerCorrecteur(float erreur, float dt_s, bool reinitDerivee);
174
175
176 // Appelée à chaque passage d'aimant devant le capteur à effet Hall
177 void hallISR() {
178     uint32_t t_us = micros(); // timestamp en microseconde au passage de l'aimant
179
180     // Anti-rebond
181     if (hallLastPulseTime_us != 0 && (t_us - hallLastPulseTime_us < 20000UL)) return;
182
183     hallPulseCount++; // incrémenter le compteur total d'impulsions (méthode fréquentométrique)
184
185     if (hallLastPulseTime_us != 0) { // si pas la toute première impulsion
186         hallPeriode_us = t_us - hallLastPulseTime_us; // durée entre les deux derniers aimants
187         hallNouvelleImpulsion = true; // signalement nouvelle mesure dispo
188     }
189
190     hallLastPulseTime_us = t_us; // mémoriser l'heure de cette impulsion pour le prochain calcul de période
191 }
192
193 // logique du bouton
194 void gererBoutonStartStop(uint32_t now) {
195     int rawState = digitalRead(BUTTON_PIN); // lire l'état du bouton (HIGH ou LOW)
196
197     if (rawState != buttonLastRawState) { // état différent d'avant ?
198         buttonLastRawState = rawState; // mémoriser le nouvel état
199         lastButtonChangeTime = now; // démarrer le chrono de l'anti-rebond
200     }
201
202     // si l'état reste le même pendant 30 ms, on valide l'état
203     if ((now - lastButtonChangeTime) >= DEBOUNCE_MS && rawState != buttonStableState) {
204         buttonStableState = rawState; // valider le nouvel état stable
205     }
206
207     if (buttonStableState == BUTTON_ACTIVE_LEVEL) { // bouton pressé ?

```

```

208     programmeActif = !programmeActif; // changer l'état du programme
209
210     if (programmeActif) { // on vient de démarrer ?
211         Serial.println(F("Bouton : START"));
212         if (!sdOk) initSD(); // ouvrir un nouveau fichier CSV si la SD n'est pas encore prête
213         lastControlTime = now; // réinitialiser le timer de contrôle pour éviter un grand dt au premier cycle
214         lastLogTime = now; // réinitialiser le timer de log
215     } else { // on vient de s'arrêter ?
216         Serial.println(F("Bouton : STOP"));
217         arreterProgramme(); // appelle arreterProgramme()
218     }
219 }
220 }
221 }
222
223 // remet toutes les variables d'état à zéro et ferme le fichier
224 void arreterProgramme() {
225     appliquerCommande(0.0f); // arrête le moteur
226     derniereCommande = 0.0f; // effacer la dernière commande mémorisée
227     integrale = 0.0f; // remettre l'intégrale à zéro
228     vitessePeriodePrecedente = 0.0f; // effacer la vitesse précédente (pour D)
229     deriveeInitialisee = false; // indiquer que D doit être réinitialisé au prochain démarrage
230     kickActif = false; // annuler le kick si il y en a un en cours
231     kickDebutMs = 0; // effacer le timestamp du kick
232     commandeLisseePWM = 0.0f; // remettre la rampe à zéro
233     fermerLogSD(); // fermeture du fichier
234
235     noInterrupts(); // désactiver les interruptions pour lire les variables volatiles sans conflit
236     lastHallCount = hallPulseCount; // remettre le compteur fréquentométrique à niveau
237     hallLastPulseTime_us = 0; // effacer le timestamp Hall (forcer vitesse nulle pour après)
238     hallPeriode_us = 0; // effacer la période Hall
239     hallNouvelleImpulsion = false; // effacer le drapeau de nouvelle impulsion
240     interrupts(); // réactiver les interruptions
241
242     vitesse_periode_ms = 0.0f; // remettre la vitesse zéro
243 }
244
245 // mesure la distance jusqu'à l'obstacle devant la voiture
246 float lireDistance() {
247     digitalWrite(TRIG_PIN, LOW); // s'assurer que TRIG est bas avant de démarrer
248     delayMicroseconds(2); // attendre 2 µs pour stabiliser le signal
249     digitalWrite(TRIG_PIN, HIGH); // envoyer l'impulsion ultrason de début
250     delayMicroseconds(10); // maintenir l'impulsion 10 µs (durée minimale pour le HC-SR04)
251     digitalWrite(TRIG_PIN, LOW); // arrêter l'impulsion
252
253     // Mesurer la durée de l'écho, si timeout -> +2 m
254     long duree_us = pulseIn(ECHO_PIN, HIGH, ECHO_TIMEOUT_US);
255
256     if (duree_us == 0) return -1.0f; // si durée nulle : pas d'écho reçu (timeout ou obstacle hors portée)
257
258     // vitesse du son = 343 m/s = 0.000343 m/µs -> 0.0001715 m/µs (divisé par 2 car aller-retour)
259     float d = (float)duree_us * 0.0001715f;
260
261     if (d < 0.02f || d > 4.0f) return -1.0f; // valeur hors plage : rejetée
262
263     return d; // renvoie la distance en mètres
264 }
265
266 /* méthode fréquentométrique de mesure de vitesse
267 compte les impulsions Hall sur 50 ms
268 peu précis à basse vitesse, très précis à haute vitesse
269 utilisé uniquement pour le log ici */
270 float calculerVitesse(uint32_t dt_ms) {
271     noInterrupts(); // bloquer les interruptions pour lire hallPulseCount (volatile) sans conflit
272     uint32_t cnt = hallPulseCount; // récupérer le nombre d'impulsions totales
273     interrupts(); // rétablir les interruptions
274
275     uint32_t delta = cnt - lastHallCount; // nombre d'impulsions depuis le dernier appel
276     lastHallCount = cnt; // mémoriser la valeur actuelle pour le prochain cycle
277
278     if (dt_ms == 0) return 0.0f; // protection contre division par zéro
279
280     // v = (nombre_de_tours * périmètre) / durée
281     // nombre_de_tours = delta / NB_AIMANTS (3 aimants = 3 impulsions par tour)
282     return (((float)delta / (float)NB_AIMANTS) * PERIMETRE_M) / ((float)dt_ms / 1000.0f);
283 }
284
285 /* méthode période de mesure de vitesse
286 mesure la durée entre deux fronts montants
287 plus précis à basse vitesse
288 incertitude constante
289 filtre exponentiel asymétrique pour lisser sans retarder */
290 float calculerVitessePeriode() {
291     // pas = périmètre / nb_aimants
292     static const float PAS_M = PERIMETRE_M / (float)NB_AIMANTS;
293
294     // Variation maximale physiquement possible en un cycle de 50 ms.
295     // Au-delà, c'est un parasite (rebond mécanique) -> on rejette.
296     static const float DELTA_V_MAX = 0.8f; // m/s par cycle de 50 ms
297
298     noInterrupts();
299     uint32_t derniereImp_us = hallLastPulseTime_us; // lire le timestamp de la dernière impulsion
300     interrupts();
301
302     if (derniereImp_us == 0) return 0.0f; // pas encore d'impulsion reçue depuis le démarrage
303
304     // Si aucune impulsion depuis plus de 400 ms -> roue à l'arrêt
305     if ((micros() - derniereImp_us) > HALL_TIMEOUT_US) {
306         noInterrupts(); hallNouvelleImpulsion = false; interrupts(); // effacer le drapeau
307         vitesse_periode_ms = 0.0f; // forcer la vitesse à zéro
308     }
309 }

```

```

312     return 0.0f;
313 }
314
315 noInterrupts();
316 uint32_t periode_us = hallPeriode_us; // copier la période mesurée par l'ISR
317 hallNouvelleImpulsion = false; // effacer le drapeau de nouvelle mesure
318 interrupts();
319
320 if (periode_us == 0) return vitesse_periode_ms; // pas de nouvelle période -> garder l'ancienne valeur
321
322 // v = distance / temps = PAS_M / (periode_us / 1 000 000)
323 float v_brute = PAS_M / ((float)periode_us / 1000000.0f);
324
325 // Rejet des parasites : si le saut de vitesse est physiquement impossible -> ignorer
326 if (vitesse_periode_ms > 0.0f
327     && fabsf(v_brute - vitesse_periode_ms) > DELTA_V_MAX) {
328     return vitesse_periode_ms; // mesure rejetée, on conserve la valeur précédente
329 }
330
331 // Filtre exponentiel asymétrique :
332 // montée -> alpha = 0.25 (filtre fort : atténue les fausses vitesses élevées)
333 // descente -> alpha = 0.85 (filtre faible : suit vite la décélération réelle)
334 // v_filtré = alpha * v_brute / (1-alpha) * v_filtré_précédent
335 float alpha = (v_brute >= vitesse_periode_ms) ? 0.25f : 0.85f;
336
337 if (vitesse_periode_ms == 0.0f) { // premier calcul : initialiser directement sans filtrage
338     vitesse_periode_ms = v_brute;
339 } else {
340     vitesse_periode_ms = alpha * v_brute + (1.0f - alpha) * vitesse_periode_ms; // filtre
341 }
342
343 return vitesse_periode_ms; // retourner la vitesse filtrée en m/s
344 }
345
346 // envoi PWM au moteur
347 void appliquerCommande(float cmd) {
348     // Zone morte : entre 0 et 76 PWM le moteur ne produit aucun couple.
349     // On force à 0 pour éviter que l'intégrale accumule une erreur fictive.
350     if (cmd > 0.0f && cmd < PWM_ZONE_MORTE) cmd = 0.0f;
351
352     int pwm = (int)constrain(cmd, -255.0f, 255.0f); // limiter entre -255 et +255 et convertir en entier
353
354     if (pwm >= 0) { // commande positive -> sens avant
355         digitalWrite(IN3_PIN, HIGH); // IN3 = HIGH : sélectionne le sens avant
356         digitalWrite(IN4_PIN, LOW); // IN4 = LOW : complément de IN3
357         analogWrite(ENB_PIN, (uint8_t)pwm); // ENB : fixer la puissance par PWM (0...255)
358     } else { // commande négative -> freinage actif / sens arrière
359         digitalWrite(IN3_PIN, LOW); // IN3 = LOW : sens inverse
360         digitalWrite(IN4_PIN, HIGH); // IN4 = HIGH : complément de IN3
361         analogWrite(ENB_PIN, (uint8_t)(-pwm)); // valeur absolue pour PWM (toujours positif 0...255)
362     }
363 }
364
365 // calcule la commande à partir de l'erreur en la faisant passer dans le correcteur
366 float calculerCorrecteur(float erreur, float dt_s, bool reinitDerivee) {
367     // terme P
368     float u_P = Kp_VAL * erreur; // proportionnel : réponse immédiate proportionnelle à l'écart courant
369
370     // terme I
371     float u_I = 0.0f; // initialiser à 0 ; sera calculé seulement si USE_I = 1
372     #if USE_I // bloc compilé seulement si le correcteur a une intégrale (PI ou PID)
373     if (!kickActif) { // NE PAS intégrer pendant le kick (anti-windup kick)
374         integrale += erreur * dt_s; // intégrale discrète : I += eps * delta_t (approximation rectangulaire)
375         integrale = constrain(integrale, // saturation anti-windup : limiter l'intégrale à + ou - INTEGRALE_MAX
376                               -INTEGRALE_MAX,
377                               INTEGRALE_MAX);
378     }
379     u_I = Ki_VAL * integrale; // terme intégral en PWM = Ki * integrale(eps*dt)
380     #endif
381
382     // terme D (sur la mesure pas sur l'erreur)
383     float u_D = 0.0f; // initialiser à 0 ; sera calculé seulement si USE_D = 1
384     #if USE_D // bloc compilé seulement si le correcteur a une dérivée (PD ou PID)
385     if (!reinitDerivee && deriveeInitialisee && dt_s > 0.0f) {
386         // dv/dt ~ (v_actuel - v_précédent) / delta_t (dérivée numérique 1er ordre)
387         float dv_dt = (vitesse_periode_ms - vitessePeriodePrecedente) / dt_s;
388         // u_D = -Kd * dv/dt car d(erreur)/dt = -dv/dt (Vc constant en régime)
389         u_D = -Kd_VAL * dv_dt;
390     }
391     // reinitDerivee = vrai lors d'une transition de mode -> on saute ce cycle
392     // deriveeInitialisee = faux au démarrage ou après un kick -> pas de "précédent" disponible
393     #endif
394
395     vitessePeriodePrecedente = vitesse_periode_ms; // mémoriser la vitesse pour le prochain calcul D
396     deriveeInitialisee = true; // indiquer que le "précédent" est maintenant valide
397
398     return u_P + u_I + u_D; // retourner la somme des trois termes PID (en PWM)
399 }
400
401 /* calcule la commande du moteur à partir de la commande du correcteur
402 et applique la logique du kick-start et la rampe asymétrique */
403 float calculerCommandeMoteur(float cmdCorrecteur, float dt_s, bool urgence, bool interdireKick, uint32_t now) {
404     static uint32_t kickFinMs = 0; // timestamp de la fin du dernier kick
405     const uint16_t KICK_GRACE = 300; // délai de grâce après un kick : 300 ms sans nouveau kick
406
407     // mode urgence, bypass tout
408     if (urgence) {
409         kickActif = false; // couper tout kick en cours
410         commandeLisseePWM = cmdCorrecteur; // mettre la rampe à jour
411     }

```

```

416     return cmdCorrecteur; // retourner la commande d'urgence sans rampe ni kick
417 }
418
419 if (interdireKick) kickActif = false; // si obstacle proche -> couper le kick immédiatement
420
421 // Kick en cours : continuer tant que conditions satisfaites
422 if (kickActif) {
423     if ((now - kickDebutMs) < KICK_DUREE_MS_C // kick encore dans sa durée maximale ?
424         && vitesse_periode_ms < VITESSE_QUASI_NULLE_MS) { // ET roue encore quasi arrêtée ?
425         commandeLisseePWM = KICK_PWM_C; // maintenir la rampe au niveau du kick
426         return (float)KICK_PWM_C; // envoyer la commande kick directement (sans rampe)
427     } else { // fin du kick (durée écoulée OU voiture a démarré)
428         kickActif = false; // désactiver le drapeau kick
429         kickFinMs = now; // mémoriser l'heure de fin pour le délai de grâce
430         deriveeInitialisee = false; // réinitialiser D : évite le pic delta_v au premier cycle post-kick
431         integrale = 0.0f; // réinitialiser I : évite le windup accumulé pendant le kick
432         commandeLisseePWM = FEEDFORWARD_PWM; // repartir la rampe depuis le feedforward (transition douce)
433     }
434 }
435
436 // déclenchement d'un nouveau kick
437 bool graceActive = (now - kickFinMs) < KICK_GRACE; // vrai si on est dans les 300 ms post-kick
438 bool roueArretee = (vitesse_periode_ms < VITESSE_QUASI_NULLE_MS) // mesure période < 0.05 m/s
439                 && (vitesse_ms < VITESSE_QUASI_NULLE_MS); // ET mesure fréq < 0.05 m/s
440
441 if (!interdireKick // kick non interdit (pas d'obstacle proche) ?
442     && !graceActive // délai de grâce écoulé (évite les kicks en rafale) ?
443     && cmdCorrecteur > COMMANDE_MIN_DEMARRAGE // le correcteur demande bien d'avancer ?
444     && roueArretee) { // les deux mesures confirment que la roue est arrêtée ?
445     integrale = 0.0f; // reset I au lancement kick (anti-windup)
446     deriveeInitialisee = false; // reset D (v=0 -> pas de "précédent" fiable)
447     kickActif = true; // activer le kick
448     kickDebutMs = now; // mémoriser l'heure de début
449     commandeLisseePWM = KICK_PWM_C; // initialiser la rampe au niveau kick
450     return (float)KICK_PWM_C; // retourner la commande kick immédiatement
451 }
452
453 // rampe asymétrique
454 float cible = constrain(cmdCorrecteur, 0.0f, 255.0f); // limiter la cible à [0, 255] PWM
455
456 // calcul du déplacement maximal autorisé ce cycle selon la direction
457 float dMax = (cible < commandeLisseePWM) // si on descend (frein ou décélération)
458             ? RAMPE_DESCENTE_PAR_S * dt_s // descente rapide : 700 PWM/s * delta_t
459             : RAMPE_MONTÉE_PAR_S * dt_s; // montée lente : 350 PWM/s * delta_t
460
461 float delta = constrain(cible - commandeLisseePWM, -dMax, dMax); // delta limité à + ou - dMax
462 commandeLisseePWM += delta; // avancer la rampe d'un pas vers la cible
463
464 return constrain(commandeLisseePWM, 0.0f, 255.0f); // retourner la commande lissée, bornée [0,255]
465 }
466
467 // initialisation
468 void setup() {
469     Serial.begin(115200); // démarrer le port série à 115 200 baud pour les messages de debug
470
471     Serial.print(F("=== AAC TIPE v19 - correcteur : ")); // afficher la version et le correcteur actif
472     Serial.print(MODE_STR); // afficher "P", "PI", "PD" ou "PID" selon la sélection
473     Serial.println(F(" ==="));
474
475     // capteur ultrason HC-SR04
476     pinMode(TRIG_PIN, OUTPUT); // TRIG est une sortie : on génère l'impulsion
477     pinMode(ECHO_PIN, INPUT); // ECHO est une entrée : on mesure le retour
478     digitalWrite(TRIG_PIN, LOW); // s'assurer que TRIG est bas au démarrage
479
480     // capteur à effet Hall
481     pinMode(HALL_PIN, INPUT_PULLUP); // activer la résistance de pull-up interne (signal actif LOW)
482     // Attacher l'ISR hallISR() à l'interruption INT0 (broche 2) sur front descendant (FALLING)
483     attachInterrupt(digitalPinToInterrupt(HALL_PIN), hallISR, FALLING);
484
485     // bouton start/stop
486     pinMode(BUTTON_PIN, INPUT_PULLUP); // pull-up interne : bouton appuyé = LOW
487     buttonLastRawState = digitalRead(BUTTON_PIN); // lire l'état initial pour éviter un faux démarrage
488     buttonStableState = buttonLastRawState; // initialiser l'état stable = état initial
489
490     // pont en H L298N
491     pinMode(ENB_PIN, OUTPUT); // Enable B : sortie PWM pour la vitesse
492     pinMode(IN3_PIN, OUTPUT); // IN3 : sortie numérique pour le sens
493     pinMode(IN4_PIN, OUTPUT); // IN4 : sortie numérique pour le sens (complément de IN3)
494     analogWrite(ENB_PIN, 0); // démarrer avec PWM = 0 -> moteur arrêté
495     digitalWrite(IN3_PIN, LOW); // IN3 bas
496     digitalWrite(IN4_PIN, LOW); // IN4 bas -> pas de courant dans le moteur
497
498     // carte SD via SPI
499     pinMode(SD_CS_PIN, OUTPUT); // CS de la SD en sortie
500     digitalWrite(SD_CS_PIN, HIGH); // SD désélectionnée au démarrage
501     pinMode(10, OUTPUT); // pin SPI maître nécessaire même si non utilisé comme CS
502     digitalWrite(10, HIGH); // mettre HIGH pour ne pas interférer avec le bus SPI
503
504     // calcul du feedforward
505     // Formule : U_FF = (Vc / K_moteur + seuil_démarrage) * 255
506     // Cette valeur est la commande qui maintient exactement Vc en régime permanent
507     // si le modèle moteur (K, seuil) est correct.
508     // On la sature à 250 pour garder 5 PWM de marge pour le terme P.
509     FEEDFORWARD_PWM = constrain(
510         (VITESSE_CIBLE_MS / K_MOTEUR + SEUIL_DEMARRAGE) * 255.0f, // calcul brut
511         0.0f, 250.0f); // saturation à 250 PWM maxi
512
513     // Afficher un résumé des paramètres sur le port série pour vérification avant démarrage
514     Serial.print(F("Vc = ")); Serial.print(VITESSE_CIBLE_MS, 2); Serial.println(F(" m/s"));
515     Serial.print(F("U_FF = ")); Serial.print(FEEDFORWARD_PWM, 1); Serial.println(F(" PWM"));
516     Serial.print(F("Kp_BRAKE = ")); Serial.print(KP_BRAKE_PWM, 1); Serial.println(F(" PWM/m"));
517     Serial.print(F("Kp=")); Serial.print(Kp_VAL, 0);
518     Serial.print(F(" Ki=")); Serial.print(Ki_VAL, 0);
519 }

```



```

520 Serial.print(F(" Kd=")); Serial.print(Kd_VAL, 0);
521 Serial.print(F(" KICK=")); Serial.print(KICK_PWM_C);
522 Serial.print(F(" PWM / ")); Serial.print(KICK_DUREE_MS_C);
523 Serial.println(F(" ms"));
524
525 initSD(); // tenter d'ouvrir un nouveau fichier CSV sur la carte SD
526 lastControlTime = millis(); // initialiser le timer de contrôle à l'instant présent
527 lastLogTime = millis(); // initialiser le timer de log à l'instant présent
528 }
529
530 /* boucle principale
531 log : toutes les 100 ms
532 correcteur : toutes les 50 ms */
533 void loop() {
534   uint32_t now = millis(); // lire l'horloge système en millisecondes (débordement tous les 49 jours)
535
536   gererBoutonStartStop(now); // vérifier l'état du bouton et décider start/stop
537
538   if (!programmeActif) { // si le programme est à l'arrêt :
539     appliquerCommande(0.0f); // forcer le moteur à 0 en permanence (sécurité)
540     lastControlTime = now; // maintenir le timer à jour pour éviter un grand dt au démarrage
541     lastLogTime = now; // idem pour le timer de log
542     return; // sortir immédiatement de loop() sans rien calculer
543   }
544
545   // bloc de contrôle (toutes les 50 ms)
546   if (now - lastControlTime >= CONTROL_PERIOD_MS) {
547     uint32_t dt_ms = now - lastControlTime; // temps réel écoulé depuis le dernier cycle (peut différer de 50 ms)
548     lastControlTime = now; // mettre à jour le timestamp de référence
549     float dt_s = (float)dt_ms / 1000.0f; // convertir en secondes pour les calculs PID
550
551     // étape 1 : mesures
552     vitesse_ms = calculerVitesse(dt_ms); // vitesse fréquentométrique (pour log seulement)
553     vitesse_periode_ms = calculerVitessePeriode(); // vitesse période filtrée (utilisée par le PID)
554     float dist_m = lireDistance(); // distance ultrason en mètres (-1 si invalide)
555
556     // étape 2 : mémoire d'obstacle
557     // Le HC-SR04 renvoie parfois -1 (timeout) même avec un obstacle présent.
558     // Sans mémoire, un seul -1 ferait basculer en mode vitesse pour 50 ms,
559     // remettant l'intégrale à 0, puis retour en mode distance -> instabilité.
560     // Solution : si une mesure invalide arrive dans les 300 ms suivant une
561     // mesure valide, on réutilise la dernière valeur valide.
562     static float derniereDist_m = -1.0f; // dernière distance valide mémorisée
563     static uint32_t dernierDistTime_ms = 0; // timestamp de la dernière mesure valide
564     const uint16_t OBSTACLE_MEMORY_MS = 300; // durée de validité de la mémoire d'obstacle
565
566     if (dist_m >= 0.0f) { // si la mesure est valide
567       derniereDist_m = dist_m; // mettre à jour la mémoire
568       dernierDistTime_ms = now; // mémoriser l'heure
569     }
570
571     float dist_eff = dist_m; // commencer avec la mesure brute
572     if (dist_m < 0.0f // mesure invalide ?
573         && derniereDist_m >= 0.0f // on avait une valeur mémorisée ?
574         && derniereDist_m < DIST_MAX_OBSTACLE_M // cette valeur signalait un obstacle ?
575         && (now - dernierDistTime_ms) < OBSTACLE_MEMORY_MS) { // la mémoire n'a pas expiré ?
576       dist_eff = derniereDist_m; // utiliser la valeur mémorisée à la place de -1
577     }
578
579     // étape 3 : détermination du mode
580     // 3 modes exclusifs, par ordre de priorité décroissante :
581     // mode 1 – urgence : obstacle très proche -> freinage maximal immédiat
582     // mode 2 – vitesse : pas d'obstacle -> maintenir Vc
583     // mode 3 – distance : obstacle en zone ACC -> maintenir vitesse + freiner si trop proche
584     uint8_t modeActuel;
585
586     if (dist_eff > 0.0f && dist_eff < DIST_URGENCE_M + 0.5f * vitesse_periode_ms) {
587       // urgence : obstacle détecté ET sa distance est inférieure à 6 cm + marge cinétique
588       // la marge 0.5*v permet d'anticiper l'urgence si la voiture roule vite
589       modeActuel = 1;
590     } else if (dist_eff < 0.0f || dist_eff >= DIST_MAX_OBSTACLE_M) {
591       // vitesse : pas d'obstacle détecté (dist=-1) OU obstacle trop loin (>1.20 m)
592       modeActuel = 2;
593     } else {
594       // distance : obstacle présent dans la zone [0.06 m, 1.20 m] -> ACC actif
595       modeActuel = 3;
596     }
597
598     // étape 4 : interdire kick si obstacle proche
599     bool interdireKick = (modeActuel == 3) // en mode distance : toujours interdit
600       || (dist_eff > 0.0f && dist_eff < DIST_CONSIGNE_M); // obstacle dans la zone de sécurité
601
602     // étape 5 : transition entre les modes
603     static uint8_t ancienMode = 0; // mode du cycle précédent (persistant entre les appels)
604     bool reinitDerivee = false; // drapeau : réinitialiser le terme D ce cycle ?
605
606     if (modeActuel != ancienMode) { // si le mode vient de changer
607       reinitDerivee = true; // toujours réinitialiser la dérivée lors d'une transition
608       // (évite un saut sur le terme D à cause du changement d'erreur)
609
610       // réinitialiser l'intégrale SEULEMENT en cas de transition depuis/vers l'urgence.
611       // garder l'intégrale évite le saut de commande
612       if (modeActuel == 1 || ancienMode == 1) {
613         integrale = 0.0f; // reset I lors d'une transition urgence depuis/vers autre mode
614       }
615     }
616     ancienMode = modeActuel; // mémoriser le mode pour la comparaison du prochain cycle
617
618     // étape 6 : calcul de la commande
619     float erreur = 0.0f; // erreur de vitesse eps = Vc - v_mesurée (initialisée à 0)
620     float commande = 0.0f; // commande PWM à envoyer (initialisée à 0)
621
622     if (modeActuel == 1) {
623       // mode 1 : urgence

```

```

624 integrale = 0.0f; // vider l'intégrale -> pas d'effet résiduel sur le freinage
625 deriveeInitialisee = false; // invalider le terme D (v peut chuter brutalement)
626 erreur = VITESSE_CIBLE_MS - vitesse_periode_ms; // calculer l'erreur pour le log uniquement
627 commande = -255.0f; // freinage maximal : -255 PWM -> L298N en sens inverse
628
629 } else if (modeActuel == 2) {
630 // mode 2 : vitesse (régulation de croisière)
631 // erreur de vitesse : positive si on est trop lent, négative si trop rapide
632 erreur = VITESSE_CIBLE_MS - vitesse_periode_ms;
633 // calculer la correction PID
634 float corr = calculerCorrecteur(erreur, dt_s, reinitDerivee);
635 // commande = feedforward + correction PID
636 commande = FEEDFORWARD_PWM + corr;
637
638 } else {
639 // mode 3 : distance
640 erreur = VITESSE_CIBLE_MS - vitesse_periode_ms; // même erreur de vitesse que mode 2
641 float corr = calculerCorrecteur(erreur, dt_s, reinitDerivee); // même PID que mode 2
642 float cmd_vitesse = FEEDFORWARD_PWM + corr; // commande qu'on aurait en mode vitesse
643
644 // terme de freinage : proportionnel à l'excès de proximité par rapport à Dc
645 float freinage = KP_BRAKE_PWM * max(0.0f, DIST_CONSIGNE_M - dist_eff);
646
647 commande = max(0.0f, cmd_vitesse - freinage); // soustraire le freinage, jamais négatif ici
648
649 // anti-windup freinage
650 #if USE_I // ce bloc n'est compilé que pour les correcteurs avec intégrale (PI et PID)
651 if (freinage > 0.0f) {
652 integrale -= erreur * dt_s; // soustraire ce qu'on vient d'ajouter dans calculerCorrecteur()
653 integrale = constrain(integrale, -INTEGRALE_MAX, INTEGRALE_MAX); // re-saturer par sécurité
654 }
655 #endif
656 }
657
658 // étape 7 : rampe + kick + zone morte
659 bool urgence = (modeActuel == 1); // drapeau urgence pour calculerCommandeMoteur()
660
661 // appliquer le kick-start et la rampe asymétrique sur la commande calculée
662 commande = calculerCommandeMoteur(commande, dt_s, urgence, interdireKick, now);
663
664 // saturation finale à [-255, +255]
665 commande = constrain(commande, -255.0f, 255.0f);
666
667 // anti-windup zone morte
668 if (commande > 0.0f && commande < PWM_ZONE_MORTE) {
669 integrale -= erreur * dt_s; // annuler l'intégrale accumulée inutilement
670 integrale = constrain(integrale, -INTEGRALE_MAX, INTEGRALE_MAX); // re-saturer
671 }
672
673 derniereCommande = commande; // mémoriser pour le log
674 appliquerCommande(commande); // envoyer au pont en H -> moteur
675
676 // étape 8 : log csv
677 if (now - lastLogTime >= LOG_PERIOD_MS) {
678 lastLogTime = now; // mettre à jour le timer de log
679
680 // en mode vitesse, log dist = -1 (pas d'obstacle, distance non pertinente)
681 float dist_log = (modeActuel == 2) ? -1.0f : dist_eff;
682
683 // écrire une ligne CSV sur la carte SD
684 ecrireLog((float)now / 1000.0f, dist_log, erreur, modeActuel);
685 }
686
687 // étape 9 : debug console
688 // afficher l'état du système sur le moniteur série à chaque cycle de contrôle (50 ms)
689 Serial.print(F("[I]")); Serial.print(MODE_STR); Serial.print(F(" ")); // [P], [PI], [PD] ou [PID]
690
691 Serial.print(F("mode="));
692 if (modeActuel == 1) Serial.print(F("URGENCE ")); // mode urgence : freinage
693 else if (modeActuel == 2) Serial.print(F("VITESSE ")); // mode vitesse : croisière
694 else Serial.print(F("DIST ")); // mode distance : ACC actif
695
696 Serial.print(F("| v=")); Serial.print(vitesse_periode_ms, 3); // vitesse mesurée (méthode période)
697 Serial.print(F(" | d=")); Serial.print(dist_m, 3); // distance brute du capteur ultrason
698 Serial.print(F(" | e=")); Serial.print(erreur, 3); // erreur de vitesse eps = Vc - v
699 Serial.print(F(" | cmd=")); Serial.print(commande, 1); // commande envoyée au moteur (PWM)
700
701 if (modeActuel == 3) { // en mode distance : afficher aussi la valeur du freinage
702 float fr = KP_BRAKE_PWM * max(0.0f, DIST_CONSIGNE_M - dist_eff); // recalculer le freinage pour l'affichage
703 Serial.print(F(" | brk=")); Serial.print(fr, 1); // valeur du freinage en PWM
704 }
705
706 Serial.print(F(" | kick=")); Serial.println(kickActif ? F("OUI") : F("non")); // état du kick
707 }
708 }
709
710 // initialisation carte SD
711 void initSD() {
712 if (!SD.begin(SD_CS_PIN)) { // tenter d'initialiser la SD sur la broche CS définie
713 Serial.println(F("SD : échec init")); // afficher l'erreur sur le port série
714 sdOk = false; // marquer la SD comme non disponible
715 return; // abandonner l'initialisation
716 }
717 }
718
719 char fn[13]; uint8_t n = 0; // nom de fichier et numéro incrémental
720 do {
721 sprintf(fn, sizeof(fn), "%s%03d.CSV", MODE_STR, n++); // générer "PID000.CSV", "PID001.CSV", etc.
722 } while (SD.exists(fn) && n < 255); // incrémenter jusqu'à trouver un nom non utilisé
723
724 logFile = SD.open(fn, FILE_WRITE); // ouvrir le fichier en écriture (crée le fichier si inexistant)
725 if (!logFile) { // si l'ouverture a échoué
726 Serial.print(F("SD : impossible d'ouvrir ")); Serial.println(fn); // afficher l'erreur
727 sdOk = false; // marquer la SD comme non disponible

```

```

728     return;
729 }
730
731 // Écrire le commentaire d'en-tête avec les paramètres de l'essai (ligne commençant par #)
732 logFile.print(F("#correcteur=")); logFile.print(MODE_STR); // nom du correcteur
733 logFile.print(F(",Kp=")); logFile.print(Kp_VAL, 1); // gain proportionnel
734 logFile.print(F(",Ki=")); logFile.print(Ki_VAL, 1); // gain intégral
735 logFile.print(F(",Kd=")); logFile.print(Kd_VAL, 1); // gain dérivé
736 logFile.print(F(",FF=")); logFile.print(FEEDFORWARD_PWM, 1); // feedforward calculé
737 logFile.print(F(",Vc=")); logFile.println(VITESSE_CIBLE_MS, 2); // vitesse cible
738
739 // Écrire la ligne d'en-tête CSV avec le nom des colonnes
740 logFile.println(F("t_s,mode,vitesse_freq_ms,vitesse_periode_ms,consigne_vitesse_ms,distance_m,consigne_dist_m,erreur_m,commande_pwm"));
741
742 logFile.flush(); // forcer l'écriture sur la carte SD (évite la perte des en-têtes en cas de coupure)
743
744 strncpy(logFilename, fn, sizeof(logFilename)); // copier le nom du fichier dans logFilename
745 logFilename[sizeof(logFilename)-1] = '\0'; // s'assurer que la chaîne est bien terminée par '\0'
746 logLinesSinceFlush = 0; // remettre le compteur de lignes depuis le dernier flush à zéro
747 sdOk = true; // marquer la SD comme opérationnelle
748
749 Serial.print(F("SD : ")); Serial.println(fn); // confirmer l'ouverture sur la console
750 }
751
752
753 // fermeture propre du fichier
754 void fermerLogSD() {
755     if (sdOk && logFile) { // si la SD est disponible et le fichier ouvert
756         logFile.flush(); // écrire toutes les données en mémoire tampon sur la carte
757         logFile.close(); // fermer proprement le fichier (finalise l'en-tête FAT)
758     }
759     sdOk = false; // marquer la SD comme indisponible jusqu'à la prochaine initSD()
760 }
761
762
763 // écris une ligne sur le fichier sur la carte SD
764 void ecrireLog(float t_s, float dist_m, float erreur_m, uint8_t mode) {
765     if (!sdOk) return; // ne rien faire si la SD n'est pas disponible
766
767     logFile.print(t_s, 3); logFile.print(','); // temps en secondes (3 décimales)
768     logFile.print(mode); logFile.print(','); // mode : 1=urgence, 2=vitesse, 3=distance
769     logFile.print(vitesse_ms, 4); logFile.print(','); // vitesse fréquentométrique (m/s)
770     logFile.print(vitesse_periode_ms, 4); logFile.print(','); // vitesse période filtrée (m/s) – utilisée par PID
771     logFile.print(VITESSE_CIBLE_MS, 4); logFile.print(','); // consigne de vitesse (m/s) – constante
772     logFile.print(dist_m, 4); logFile.print(','); // distance obstacle (-1 si mode vitesse)
773     logFile.print(DIST_CONSIGNE_M, 4); logFile.print(','); // consigne de distance (m) – constante
774     logFile.print(erreur_m, 4); logFile.print(','); // erreur de vitesse  $\epsilon = V_c - v$  (m/s)
775     logFile.println(derniereCommande, 1); // commande envoyée au moteur (PWM, newline)
776
777     // flush groupé : écrire physiquement sur la SD tous les LOG_FLUSH_EVERY_N_LINES cycles
778     // (flush à chaque ligne serait trop lent et userait la carte SD inutilement)
779     if (++logLinesSinceFlush >= LOG_FLUSH_EVERY_N_LINES) {
780         logFile.flush(); // forcer l'écriture du buffer SD sur la carte
781         logLinesSinceFlush = 0; // remettre le compteur à zéro
782     }
783 }

```